

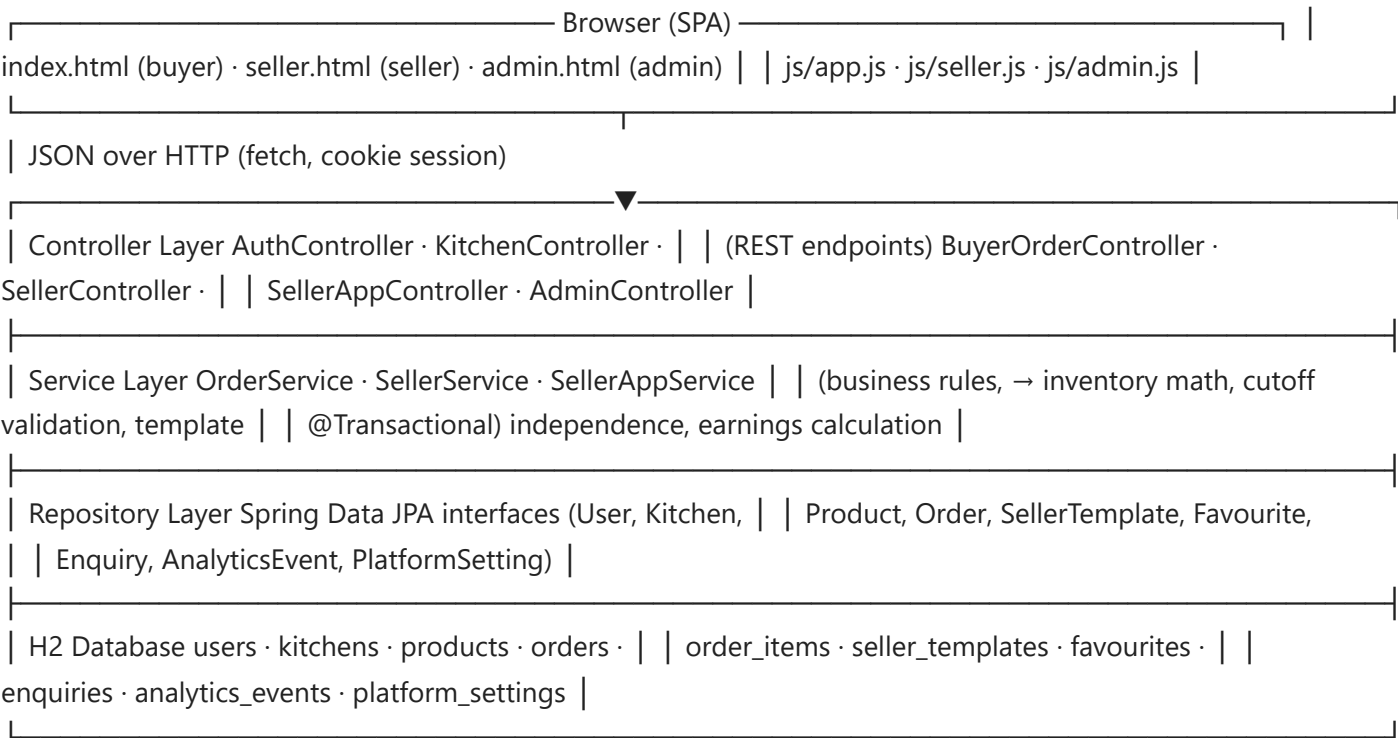
SocioMart Technical Document

1. Technology Stack

LayerTechnologyWhy It Was Chosen |---|---|---| LanguageJava 21Stable LTS, strong typing, industry standard for enterprise APIs FrameworkSpring Boot 4.1.1Fast REST development, built-in dependency injection, security, validation Data LayerSpring Data JPA + HibernateObject-relational mapping without boilerplate SQL; automatic schema from entities DatabaseH2 (in-memory by default)Relational integrity for orders/payments; reseeded with Indian food demo data on every boot Build ToolMaven (mvnw wrapper)Reproducible builds — no local install needed, works in CI FrontendHTML5 + CSS3 + Vanilla JS (SPA)Zero build step, instant load, served straight from Spring's static folder SessionsHttpSession cookiesSimple, secure login state for buyer/seller flows Public AccessRender (Docker, free tier)Free HTTPS public URL — single service serves Buyer (/) and Seller (/seller.html) with shared H2 state CIGitHub ActionsEvery push is auto-compiled so broken code can never reach main Docs & TestingEdge headless (PDF), PowerShell E2E harnessScreenshot-driven docs, regression suite

2. System Architecture

The application follows a classic 3-layer architecture. The browser never talks to the database directly — every request flows through the API layer, which enforces login and ownership rules. ``



``` Key design decision: GlobalExceptionHandler converts every business error (sold-out item, unauthorised seller, cap reached) into a clean JSON `{error, message}` response with the correct HTTP status — the frontend never crashes on an unexpected payload.

## 2.1 Buyer Route Ownership

The buyer SPA uses hash-based routing:

- `/#/kitchens` → `kitchensView()`` — the dedicated client-facing Buyer route
- `/#/kitchen/{id}` → `kitchenPageView()`` — public kitchen detail
- `/#/home`, `/#/food`, `/#/summary`, `/#/confirm`, `/#/payment`, `/#/favourites`, `/#/orders`, `/#/profile``

The raw `/kitchens`` path is publicly accessible but has backend/admin route ownership conflict and is not the final Buyer navigation path.

## 2.2 Frontend CTA Fix

`updateCartBar()`` in `static/js/common.js`` hides `#viewOrderBar`` when:

- cart has zero items
- OR
- current view contains `.sticky-footer-bar``

This prevents duplicate/overlapping Place Order CTAs on Order Summary, Confirm Order, and Payment screens.

## 3. Database Design

---

Eight tables, one per real-world concept. Relationships use JPA ManyToOne joins; demo data is seeded by DemoDataSeeder on first boot (idempotent, guarded by platform\_settings flags). TablePurposeKey Columns  
 |---|---|---| usersBuyers, sellers, adminmobile\_number (unique), name, role, is\_seller, flat\_number, society, building kitchensA seller's shopfrontseller\_id (FK), name, slug, rating, available\_today, verified, society, building, image\_url, instagram\_link productsOne day's live offeringkitchen\_id (FK), price, max\_quantity, remaining\_quantity, available\_date, cutoff\_time, ready\_by\_time, is\_preorder, preorder\_type ordersOne purchase eventbuyer\_id, kitchen\_id, total\_amount, payment\_status, order\_status, buyer\_name, buyer\_flat, order\_number order\_itemsLine items of an orderorder\_id (FK), product\_id (FK), product\_name, quantity, unit\_price seller\_templatesSaved favourites (max 3)seller\_id (FK), name, description, price, max\_quantity, cutoff\_time, order\_window\_start, order\_window\_end favouritesBuyer favourite kitchensuser\_id (FK), kitchen\_id (FK) enquiriesCustomer enquiriesuser\_id (FK), kitchen\_id (FK), message, status, created\_at analytics\_eventsRaw behaviour logkitchen\_id, event\_type (MENU\_VIEW...), created\_at platform\_settingsSeed guards and configsetting\_key (unique), setting\_value Why remaining\_quantity is its own column: the stepper updates one integer per click instead of re-deriving stock from orders — fast and race-safe. Why order\_items.product\_name is denormalised: a dish can be renamed or deleted later; the historical order must still show exactly what the buyer ordered.

## 4. REST API Reference

---

All endpoints return JSON. Errors always arrive as `{ "error": " ", "message": " " }` with a proper HTTP status (400 bad input, 401 not logged in, 403 wrong owner, 404 missing, 409 conflict).

### 4.1 Auth & Public

Method & Path	What It Does
POST /api/auth/otp/request	publicRequest OTP for mobile number
POST /api/auth/otp/verify	publicVerify OTP, create session
POST /api/auth/logout	authenticatedInvalidate session
GET /api/auth/me	authenticatedCurrent user info
GET /api/kitchens	publicPublic kitchen list
GET /api/kitchens/id/{id}	publicOne kitchen with today's offerings (logs MENU_VIEW)
GET /api/discovery/kitchens	publicKitchen discovery with tab filters
GET /api/discovery/items	publicItem discovery by category
GET /api/discovery/category-kitchens	publicKitchens by category
GET /api/discovery/counts	publicKitchen counts by tab
GET /api/marketplace	publicMarketplace data
GET /api/items	publicItems list
GET /api/search	publicSearch

### 4.2 Buyer

Method & Path	What It Does
POST /api/buyer/orders/draft	buyerPrice the cart server-side
POST /api/buyer/orders/place	buyerPlace order, decrement stock atomically
GET /api/buyer/orders/my	buyerBuyer's own orders
GET /api/buyer/orders/{id}	buyerBuyer's own order detail
PUT /api/buyer/profile	buyerUpdate buyer profile
GET /api/favourites	authenticatedList user's favourite kitchens
POST /api/favourites/kitchen/{id}/toggle	authenticatedToggle favourite kitchen (max 3)
POST /api/enquiries	authenticatedSubmit enquiry

### 4.3 Seller

Method & Path	What It Does
GET /api/seller-app/dashboard	sellerEarnings, offerings, orders, real view count
GET /api/seller-app/templates	sellerList seller's favourite templates
POST /api/seller-app/templates	sellerSave offering as template (max 3)
DELETE /api/seller-app/templates/{id}	sellerDelete template
POST /api/seller-app/templates/{id}/publish	sellerRepublish template as independent offering
POST /api/seller-app/products	sellerCreate offering (manual / quick-post confirm)
PUT /api/seller/products/{id}	sellerPartial update (price, cutoff, stock...) PATCH
POST /api/seller-app/products/{id}/inventory	sellerStepper: add/subtract stock by delta
POST /api/seller-app/products/{id}/sold-out	sellerForce mark sold out (works on unlimited

items) GET /api/seller-app/orders/summarysellerPer-dish plates + revenue GET /api/seller-app/orders/product/{id}sellerPer-buyer drill-down GET /api/seller-app/earnings sellerPending / confirmed-today / this-month GET /api/seller-app/history sellerRecent items for republish POST /api/seller-app/parse-messagesellerWhatsApp Quick Post parser (read-only) PATCH /api/seller/orders/{id}/status sellerOrder pipeline transitions (incl. cancel) GET /api/seller/kitchensellerSeller's kitchen details PUT /api/seller/kitchensellerUpdate kitchen profile

## 4.4 Admin

Method & Path | What It Does | --- | --- | --- | GET /api/admin/dashboard adminDashboard stats GET /api/admin/buyers adminList all buyers GET /api/admin/sellers adminList all sellers GET /api/admin/sellers/pending adminList pending sellers POST /api/admin/sellers/{id}/approve adminApprove seller POST /api/admin/sellers/{id}/reject adminReject seller GET /api/admin/kitchens adminList all kitchens GET /api/admin/products adminList all offerings GET /api/admin/orders adminList all orders GET /api/admin/enquiries adminList all enquiries GET /api/admin/analytics adminPlatform analytics

## 5. Security Model

---

- Role-based access: BUYER, SELLER, ADMIN, SUPER\_ADMIN
- Buyer endpoints: require BUYER role
- Seller endpoints: require SELLER role
- Admin endpoints: require ADMIN or SUPER\_ADMIN role
- Super Admin endpoints: require SUPER\_ADMIN role
- Ownership checks on every mutation: seller B cannot modify seller A's kitchen/products/orders (HTTP 403)
- ID isolation: buyer cannot access another buyer's order (HTTP 404)
- Anonymous access to admin endpoints: rejected (HTTP 401)
- Session timeout: 30 minutes
- SameSite=Lax cookie protection
- Custom UserSessionAuthorizationFilter runs before UsernamePasswordAuthenticationFilter
- Verified boundaries:
  - anonymous → Admin rejected
  - Buyer → Admin rejected
  - Seller → Admin rejected
  - Buyer → seller orders rejected

- Buyer → another buyer's order rejected
- Seller cross-owner access rejected

## 6. Testing

---

### 6.1 Unit Tests (JUnit 5)

13 tests across 3 test classes:

- `OrderServicePaymentTest` (5 tests): payment status → order status mapping
- `FavouriteServiceLimitTest` (4 tests): 3-kitchen favourite cap enforcement
- `OrderServiceValidationTest` (4 tests): cutoff, stock, duplicate order validation

### 6.2 Build Verification

- ``mvnw clean compile``: PASS (97 source files, Java 21)
- ``mvnw test``: 13/13 PASS
- ``mvnw package -DskipTests``: PASS

### 6.3 E2E Regression Suite

A PowerShell harness drives the real running application over HTTP – the same path a browser takes – and asserts behaviours. It is delta-based, so it can be re-run on a live instance any number of times without false failures.

## 7. Deployment

---

Hosting provider: Render.com – free Docker web service. The app is built from the existing Dockerfile (multi-stage Maven → JRE 21) and deployed as a Render Blueprint via `render.yaml`. One service serves both Buyer (/) and Seller (/seller.html) with a shared in-process H2 database – no split instances, no data desync. Render injects the `PORT` environment variable automatically; the app binds to ``${PORT:8081}``. Render auto-deploys on every push to main. The free tier sleeps after ~15 min of inactivity; a free uptime pinger keeps the service warm. Public URLs (single Render service):

- <https://sociomart-demo.onrender.com/> – Buyer view
- <https://sociomart-demo.onrender.com/seller.html> – Seller dashboard
- <https://sociomart-demo.onrender.com/admin.html> – Admin console
- <https://sociomart-demo.onrender.com/api/kitchens> – health probe

## 8. Known Limitations

---

- H2 in-memory database: custom data disappears on restart; demo data re-seeds on every boot (accepted V1/demo limitation)
- Free tier cold-start delay (~120s) on Render – expected demo behaviour
- No real OTP delivery (dev mode returns fixed OTP)
- Mockito self-attaching warning on JDK 21 (non-fatal)